Repository link:

https://github.com/erikalank/Systeemiohjelmointi.git

Project 1: Warmup to C and Unix programming

The program reads lines from standard input or from a file and prints them in reverse order. If an output file is provided, the reversed content is written into that file. The implementation uses getline() for reading lines dynamically and malloc()/realloc() for memory allocation. The program handles multiple error cases like invalid files, too many command-line arguments, memory allocation failures and identical input/output files.

Screenshots

Program works with input file:

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project1$ ./reverse input.txt
a file
is
this
hello
```

Program writes to output file:

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project1$ ./reverse input.txt output.txt
cat output.txt
a file
is
this
hello
```

Error handling:

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project1$ ./reverse input.txt input.txt
Input and output file must differ
```

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project1$ ./reverse a b c
usage: reverse <input> <output>
```

Project 2: Unix Utilities

My-cat:

My-cat program reads one or more files and prints their contents to standard output. The implementation uses fopen(), fgets() and fclose() for file handling. The program also checks for opening file errors and prints the requires error message if a file cannot be opened.

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ echo Hello > test.txt
echo This is my-cat test >> test.txt
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ gcc -o my-cat my-cat.c -Wall -Werror
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ ./my-cat test.txt
Hello
This is my-cat test
```

Error handling:

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ ./my-cat missing.txt
my-cat: cannot open file
```

My-grep:

The my-grep searches for a given search term from one or more files and prints matching lines. The implementation uses getline() to support long input lines and strstr() to search for the search term from each line. The program also supports reading from standard input if no files are provided.

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ echo apple > grep.txt
echo banana >> grep.txt
echo pineapple >> grep.txt
echo orange >> grep.txt
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ gcc -o my-grep my-grep.c -Wall -Werror
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ ./my-grep apple grep.txt
apple
pineapple
```

My-zip:

The my-zip program compresses files using run-length encoding (RLE). The program reads files character by character using fgetc(). Consecutive repeating characters are counted and written to standard output in binary format using fwrite(). Each compressed block contains a 4-byte integer and a single character.

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ echo aaaabbbccdddd > test.txt
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ gcc -o my-zip my-zip.c -Wall -Werror
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ ./my-zip test.txt > test.z
```

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ xxd test.z
00000000: 0400 0000 6103 0000 0062 0200 0000 6304  ....a....b....c.
00000010: 0000 0064 0100 0000 0a                   ...d.....
```

My-unzip:

The my-unzip program decompresses files created by my-zip. The implementation reads compressed binary data using fread(). The program reads the count and character pairs and prints the character the required number of times.

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ gcc -o my-unzip my-unzip.c -Wall -Werror
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Sys
teemiohjelmointi/Project2$ ./my-unzip test.z
aaaabbbccdddd
```

Project 3: Kernel Hacking

A global counter variable was added to the kernel and incremented inside the sys_read() function. A new system call getreadcount() was implemented to return the current value of this counter. The system call was added by modifying syscall.h, syscall.c, sysproc.c, user.h and usys.S. The implementation follows the structure of existing system calls such as getpid().

Successful compilation and boot of xv6 using make qemu-nox:

```
erikalankinen@LAPTOP-1554LF5R:/mnt/c/Users/Erika/OneDrive/Tiedostot/Systeemiohjelmointi/Project3/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stressfs _usertests _wc _zombie _test
```

The implementation was tested by creating a simple user program that calls getreadcount() before and after invoking read(). The results show that the counter increases correctly when read() is used.

Output of the test program showing the value returned by getreadcount():

```
Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodesta
rt 32 bmap start 58
init: starting sh
$ test
Read count: 5

Read count: 6
$
```

Reset the read counter: The system call getreadcount() takes an argument, where passing value 1 resets the counter to zero. Passing 0 returns the current value without modifying it.

```
init: starting sh
$ test
Read count: 5
a
After read: 6
After reset: 0
$ $
```